Stored DOM XSS

1. Overview

- **Vulnerability:** Stored DOM-Based Cross-Site Scripting (XSS)
- **Category:** Client-Side Web Application Security
- **Platform:** PortSwigger Web Security Academy
- **Severity:** Context-dependent
- **OWASP Category:** A03:2025 – Injection

2. Description

Stored DOM XSS occurs when attacker-controlled content is saved by an application and later processed by client-side JavaScript in an unsafe manner.

Unlike traditional reflected XSS, the malicious input may be stored on the server and delivered to users on subsequent visits. The vulnerability arises when client-side code reads the stored content and passes it to an unsafe DOM sink.

3. Vulnerability Details

The application stores user-controlled content, such as a comment or other submitted text. When a page later renders that content, JavaScript processes the data in an unsafe way, potentially interpreting it as executable HTML or script.

Affected component: The application feature that stores and displays user-submitted content.

Root cause: Untrusted stored data reaches a dangerous DOM sink without appropriate validation, sanitization, or safe rendering.

4. Proof of Concept (PoC)

1. Open the assigned PortSwigger lab.
2. Identify the feature that accepts and stores user-controlled content.
3. Submit a harmless test string and confirm that it appears when the relevant page is revisited.
4. Inspect the client-side JavaScript to identify the source of the data and the DOM sink that processes it.
5. In the authorized lab, use the lab-specific payload to verify whether JavaScript execution occurs when the stored content is rendered.

Example of an unsafe DOM operation:

element.innerHTML = storedComment;

If `storedComment` contains attacker-controlled HTML, this operation can interpret the input as markup. The example illustrates the vulnerable pattern; it is not a confirmed description of the lab's exact implementation.

5. Impact

Successful exploitation may allow an attacker to:

- Execute JavaScript in the browser of users who view the affected content.
- Modify page content or display deceptive UI elements.
- Perform actions using the victim's application session, where protections permit.
- Affect multiple users because the malicious content remains stored until removed or neutralized.

The severity depends on the affected page, user privileges, reach, and application security controls.

6. Remediation

- Render untrusted text using safe DOM APIs such as `textContent`.
- Avoid passing untrusted data to `innerHTML`, `document.write`, or other unsafe DOM sinks.
- If HTML is required, sanitize it with a reputable, maintained HTML sanitizer.
- Review client-side data flows from sources to DOM sinks.
- Validate and safely handle stored content at appropriate application boundaries.
- Add regression tests covering stored content and all relevant rendering paths.
- Use Content Security Policy as an additional protective layer.

7. References

- PortSwigger Web Security Academy: https://portswigger.net/web-security/cross-site-scripting/dom-based
- OWASP DOM Based XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html

8. Disclaimer

This report documents an authorized educational lab scenario. Testing should only be performed on systems for which explicit permission has been granted.